

Agentic AI + Data Engineering Project

Overall Progress & Final Roadmap

1. Project Objective

Build an end-to-end **Agentic AI + Data Engineering platform** using:

- Python
- PySpark
- MySQL
- Apache Airflow
- Google ADK
- MCP
- LLMs
- RAG
- Agentic orchestration

The system will process the **Olist Brazilian E-Commerce dataset** as a realistic historical/incremental data source and eventually allow AI agents to answer business questions, monitor pipelines, and retrieve supporting information.

2. What We Have Completed

Phase 1 — Existing Agentic AI Foundation

Already working before the Data Engineering extension:

```
graph TD
    User --> Agent
    Agent --> Tools["Tools / MCP"]
    Tools --> FastAPI
    FastAPI --> MySQL
```

The existing API/MCP Agentic System is working and will be preserved.

We will NOT unnecessarily modify:

```
api_server/  
mcp_server/  
existing Agentic System
```

3. Project Environment

Project:

```
D:\agentic-ai-adk
```

Virtual environment:

```
.venv
```

Important components installed/configured:

- Python 3.14.7
- Google ADK 2.8.0
- PySpark 4.0.1
- Java 17
- MySQL 8.0.42
- mysql-connector-python
- python-dotenv

MySQL:

```
Database: agent_adk_db  
User: agent_adk_user  
Host: localhost  
Port: 3306
```

Database credentials are stored in `.env`.

4. Olist Dataset

The Olist dataset was extracted into:

```
data/raw/olist/
```

Important source files:

```
olist_customers_dataset.csv
olist_orders_dataset.csv
olist_order_items_dataset.csv
olist_payments_dataset.csv
olist_products_dataset.csv
olist_sellers_dataset.csv
product_category_name_translation.csv
```

Geolocation and reviews are currently not part of the initial pipeline.

5. PySpark Pipeline

We created:

```
spark_pipeline/
├── __init__.py
├── config.py
├── schemas.py
├── extract.py
├── transform.py
├── validate.py
├── load.py
├── pipeline.py
└── batch_generator.py
```

Responsibilities:

`config.py`

Handles:

- project paths
- `.env`
- MySQL configuration
- raw/batch/processed directories

`schemas.py`

Defines explicit Spark schemas for the Olist datasets.

`extract.py`

Creates the Spark session and reads source data.

`transform.py`

Handles:

- transformations
- metadata
- analytics calculations

`validate.py`

Performs basic data-quality validation.

`load.py`

Handles:

- MySQL connection
- table creation
- PySpark → MySQL loading
- batch metadata
- watermark management
- analytics refresh

`pipeline.py`

Runs one incremental batch.

`batch_generator.py`

Creates chronological monthly batches from the historical Olist data.

6. Windows + Spark Issue

Spark initially produced:

```
HADOOP_HOME and hadoop.home.dir are unset  
Did not find winutils.exe
```

We investigated the Hadoop/Spark relationship.

PySpark 4.0.1 uses Hadoop 3.4.1 internally.

Instead of installing unofficial Windows Hadoop binaries, we changed the one-time batch generation approach so that it does not use Spark's Hadoop local filesystem writer.

This is acceptable for this project.

The Spark processing itself continues to work normally.

The warnings:

```
Could not find platform independent libraries <prefix>  
WARN Shell: Did not find winutils.exe  
WARN NativeCodeLoader
```

are currently non-blocking.

7. Batch Simulation

The historical Olist data is being used to simulate incremental arrivals.

The batch generator created chronological monthly batches under:

```
data/batches/
```

Structure:

```
data/batches/  
└─ batch_YYYYMM/  
    ├─ orders/  
    │   └─ data.csv  
    ├─ order_items/  
    │   └─ data.csv  
    └─ order_payments/  
        └─ data.csv
```

Important:

`batch_generator.py` is a **one-time preparation step**.

Airflow will NOT regenerate these batches.

8. MySQL Architecture

We created three logical groups of tables.

Core/source tables

```
customers
orders
order_items
products
sellers
order_payments
product_category_translation
```

Pipeline metadata

```
pipeline_batches
pipeline_watermarks
```

Analytics

```
monthly_sales_summary
category_performance
seller_performance
```

9. Incremental Processing

The pipeline now accepts:

```
--batch-id
```

Example:

```
python -m spark_pipeline.pipeline --batch-id batch_201609
```

The process is:

```
Batch CSV
↓
PySpark
↓
Validation
↓
Transform
↓
```

```
MySQL Core Tables
↓
Analytics Refresh
↓
Batch Metadata
↓
Watermark
```

10. Incremental Pipeline Successfully Tested

We successfully processed multiple batches.

Validated batches include:

```
batch_201609 ✓
batch_201610 ✓
batch_201612 ✓
```

`batch_201611` does not exist in the generated batch set, so we correctly skipped it rather than creating artificial data.

Example successful load:

```
Orders received:      4
Order items received: 6
Payments received:    3

orders: 4 rows loaded
order_items: 6 rows loaded
order_payments: 3 rows loaded
```

The data was independently verified from Python against MySQL.

Workbench initially appeared to show zero because its session was stale. Restarting Workbench displayed the inserted data correctly.

Therefore:

MySQL loading + commit + incremental processing are confirmed working.

11. Analytics Refresh

We added the concept of refreshing:

```
monthly_sales_summary
category_performance
seller_performance
```

after the core tables are loaded.

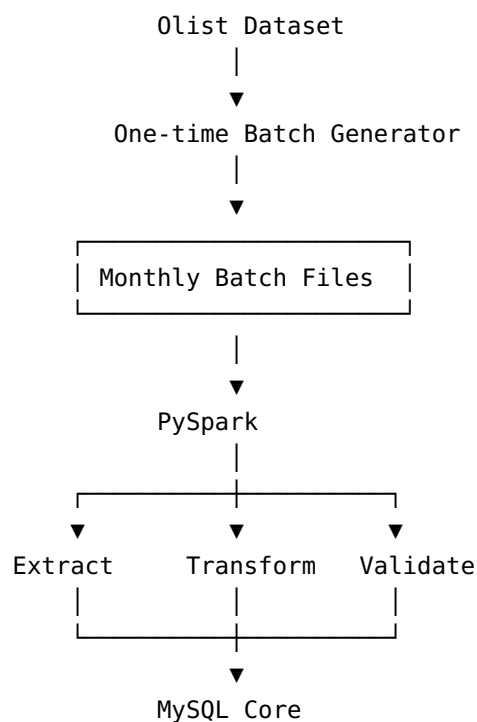
The intended sequence is:

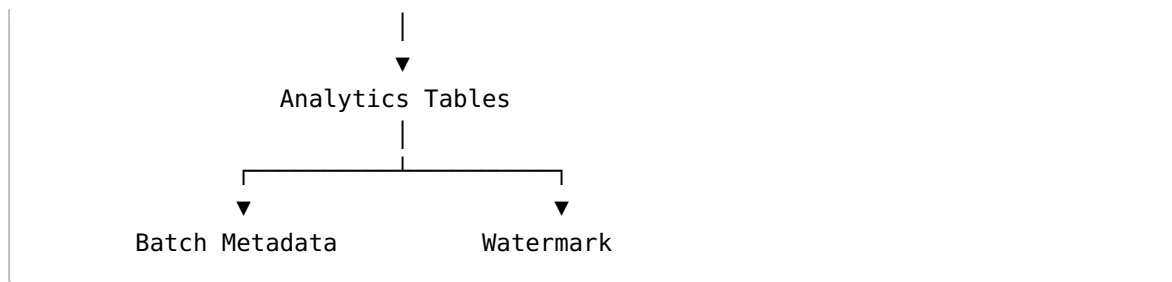
```
Load Orders
↓
Load Order Items
↓
Load Payments
↓
Refresh Analytics
↓
Mark Batch SUCCESS
↓
Update Watermark
```

This means analytics always represents all successfully loaded data rather than only the latest batch.

12. Current Architecture

At this point we have:





13. What We Are Going To Build Next

Phase 2 — Airflow Orchestration

This is the immediate next phase.

Airflow will control:

```
Find next batch
  ↓
Run PySpark pipeline
  ↓
Validate result
  ↓
Refresh analytics
  ↓
Update metadata/watermark
  ↓
Success
```

Airflow will also provide:

- scheduling
- retries
- failure handling
- task dependencies
- execution history
- monitoring

Target structure:

```
airflow/
├── dags/
│   └── olist_incremental_pipeline.py
```

14. Phase 3 — Pipeline Monitoring Agent

After Airflow is working, create:

```
agents/pipeline_monitor_agent/
```

This agent will inspect:

```
pipeline_batches  
pipeline_watermarks  
Airflow execution information
```

It should answer questions such as:

```
Is today's pipeline successful?  
  
Which batch failed?  
  
When was the last successful batch?  
  
How many records were processed?  
  
What was the latest pipeline error?
```

15. Phase 4 — Business Query Agent

Create:

```
agents/business_query_agent/
```

This agent will interact with the analytical MySQL tables.

Examples:

```
What was the revenue in September 2016?  
  
Which category generated the highest revenue?  
  
Who are the top 10 sellers?  
  
How many orders were processed?
```

What is the average order value?

Important architecture principle:

The LLM will not receive 100,000+ raw database rows.

Instead:

```
User question
  ↓
Business Query Agent
  ↓
SQL
  ↓
MySQL aggregation
  ↓
Small result
  ↓
LLM explanation
  ↓
User
```

16. Phase 5 — RAG Agent

Create:

```
agents/rag_agent/
```

The RAG layer will handle unstructured knowledge such as:

- business definitions
- data dictionary
- pipeline documentation
- schema documentation
- metric definitions
- troubleshooting documentation
- project documentation

Architecture:

```
User question
  ↓
RAG Agent
  ↓
```

```
Retriever
  ↓
Relevant documents
  ↓
LLM
  ↓
Grounded answer
```

This will complement the Business Query Agent.

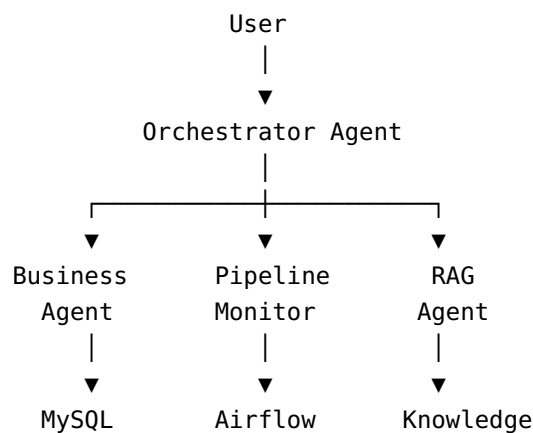
17. Phase 6 — Root Orchestrator Agent

Finally create:

```
agents/orchestrator/
```

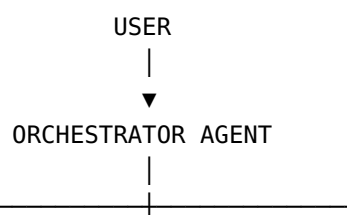
This becomes the main intelligent entry point.

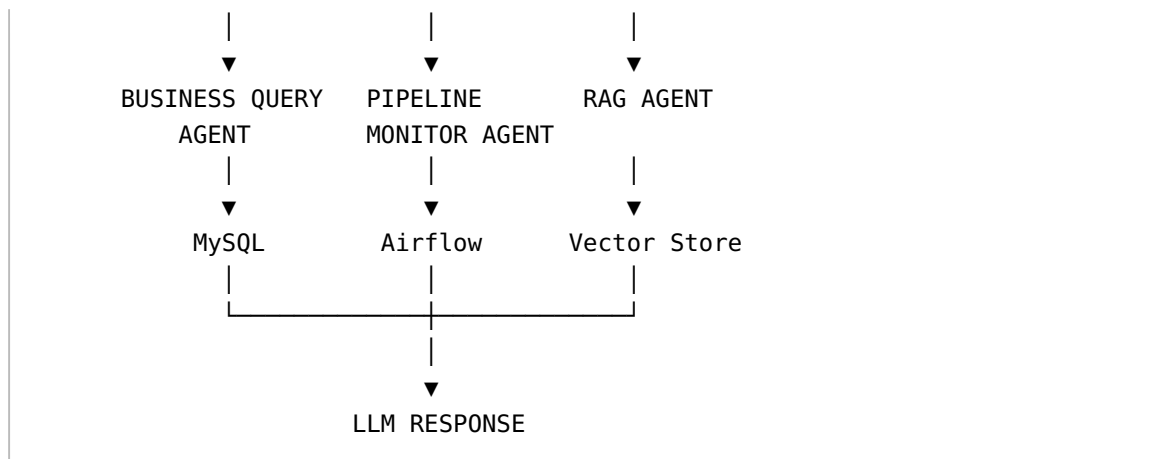
Example:



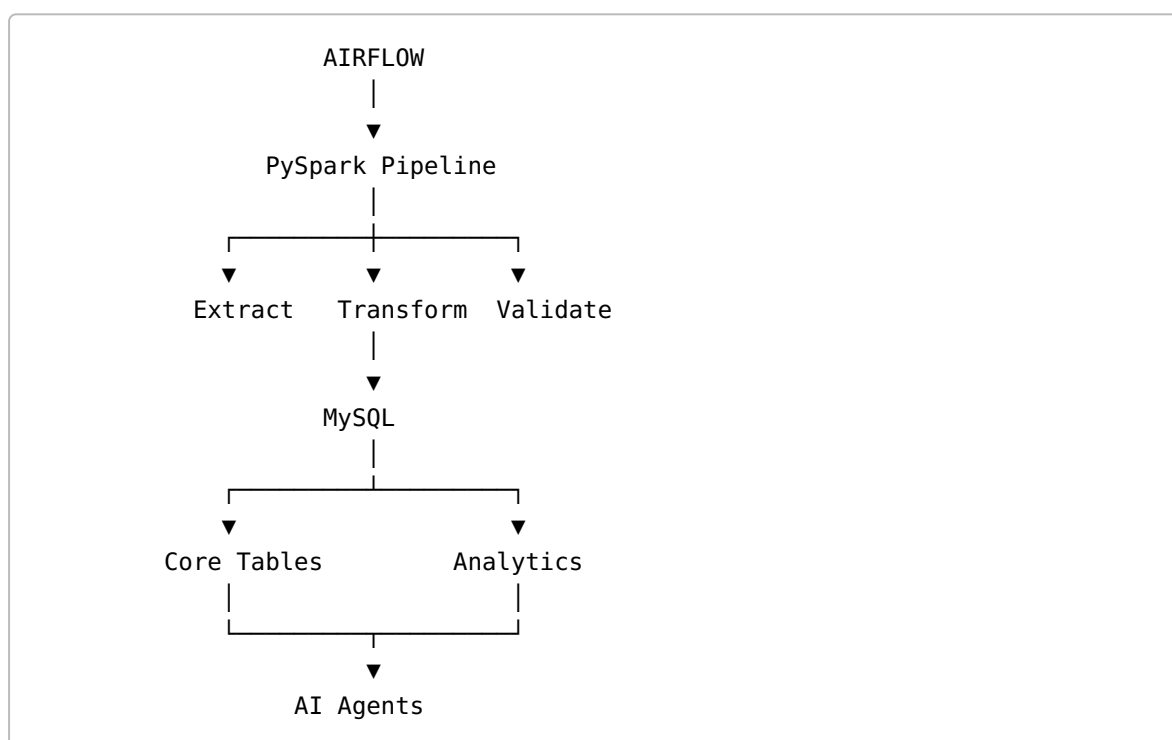
The orchestrator decides which specialist agent should handle the request.

18. Final Target Architecture





Meanwhile the data pipeline runs independently:



19. Final Project Capabilities

By the end, the project should demonstrate:

Data Engineering

- PySpark ingestion
- schema management
- transformation
- validation
- incremental processing
- batch processing

- watermarks
- MySQL storage
- analytical tables

Orchestration

- Airflow DAGs
- dependencies
- scheduling
- retries
- monitoring
- failure recovery

Agentic AI

- Google ADK
- specialist agents
- tool usage
- MCP
- agent orchestration
- business intelligence queries
- pipeline monitoring

RAG

- document ingestion
- embeddings/vector search
- retrieval
- grounded answers
- business/data documentation

20. Development Order From Here

We will follow this exact order:

- ✓ 1. Environment
- ✓ 2. Olist dataset
- ✓ 3. PySpark foundation
- ✓ 4. MySQL schema
- ✓ 5. Batch generation
- ✓ 6. Incremental pipeline
- ✓ 7. Batch testing
- ✓ 8. Analytics refresh
- ➡ 9. Airflow DAG
- ➡ 10. Airflow retries + scheduling
- ➡ 11. Pipeline Monitoring Agent
- ➡ 12. Business Query Agent
- ➡ 13. RAG Agent

- ➔ 14. Root Orchestrator
- ➔ 15. End-to-end integration
- ➔ 16. Testing
- ➔ 17. Documentation
- ➔ 18. Resume/interview/project presentation

Important project rule

We will **not restart or redesign completed components unnecessarily**.

The current working components become the foundation for the next stages.

The immediate next task is therefore:

Build the Airflow DAG that orchestrates the already-working incremental PySpark pipeline.